DECEMBER 1, 2024

APG – AMWAL WEBHOOK – PAY BY TOKEN

Amwal Pay
payment gateway

Confidential

Table of Contents

Document Version History

Versions	#	Date	Description / Modifications
	v1.0	01-12-2024	Initiation version

1 Introduction

This document outlines the process of executing payments using the Pay by Token feature of the Amwal Payment Gateway. It explains how merchants can perform secure transactions without requiring full card details by leveraging stored customer tokens. The guide also details secure hash generation with HMAC SHA-256 to protect requests and includes sample code for seamless integration.

2 Secure Hash Calculation for Using APIs

This guide provides instructions for integrating with our system using HMAC SHA256 secure hashing. Secure hashing is used to ensure the integrity and confidentiality of data transmitted between systems.

2.1 Secure Hash Generation

To ensure the security and integrity of API requests, all requests to Amwal Payment Gateway require a Secure Hash. The Secure Hash is generated by concatenating the entire JSON input as key-value pairs in alphabetical order and encrypting the resulting string using SHA-256 with the provided hash key.

Prepare the Data

- Gather all the necessary parameters required for the API request.
- Validate and sanitize the data to prevent potential security risks such as injection attacks.

Sort Parameters Alphabetically:

- Arrange all key-value pairs in alphabetical order by key.

Concatenate Key-Value Pairs:

- Format the sorted parameters as key=value pairs and join them using &.

Append Merchant Secure Key:

- Obtain your Merchant Secure Key provided by Amwal Payment Gateway and append it to the concatenated string.

Generate SHA-256 Hash:

- Use the SHA-256 hashing algorithm to encrypt the final string.
- The output hash will be the Secure Hash used for authentication.

Example input string before encryption based on the request being used:

“transactionId=ed520b67-80b2-4e1a-9b86-34208da10e53&merchantId=22914&requestDateTime=2025-02-16T12:43:51Z”

Important Note: The exact parameters used in the Secure Hash generation must match those sent in the API request. Any discrepancy will cause authentication failure.

Send the Payment Request

- Include both the original request data and the calculated Secure Hash in your API request.
- Amwal Payment Gateway will use this hash to verify request authenticity before processing.

3 Example Code

Here are some sample code snippets that demonstrate how to calculate the secure hash in different programming languages which you will need to use in every request.

```
function calcHash(obj: any, secret: string) {
  try {
    let objSorted = Object.keys(obj)
      .sort()
      .reduce(
        (acc, key) => ({
          ...acc,
          [key]: obj[key],
        }),
        {},
      );
    let dataPreparedForHashing = Object.entries(objSorted)
      .map(([key, value]) => `${key}=${value}`)
      .join('&');

    // console.log(dataPreparedForHashing) should be like this

“transactionId=ed520b67-80b2-4e1a-9b86-
34208da10e53&merchantId=22914&requestDateTime=2025-02-16T12:43:51Z”

    const hmac = crypto.createHmac('sha256', Buffer.from(secret, 'hex'));

 const     hashValue     =hmac.update(dataPreparedForHashing,     'utf-
8').digest('hex');
    return hashValue.toUpperCase();
  }
 catch (error)
 {
    return '';
  }
}
let hash = calcHash(paramsObj, "YOUR_SECRET_KEY")
```

PHP Example

```php
<?php
function encryptWithSHA256($input, $hexKey) {
    // Convert the hex key to binary
    $binaryKey = hex2bin($hexKey);
    // Calculate the SHA-256 hash using hash_hmac
    $hash = hash_hmac('sha256', $input, $binaryKey);
    return $hash;
}
    // Provided input text
    $text=""transactionId=ed520b67-80b2-4e1a-9b86-34208da10e53&merchantId=22914&requestDateTime=2025-02-16T12:43:51Z”

    // Provided hex key
    $hexKey = "9FFA1F36D6E8A136482DF921E856709226DE5A974DB2673F84DB79DA788F7E19";
    // Calculate SHA-256 hash
    $result = encryptWithSHA256($text, $hexKey);
?>
```

4 Pay by Token Endpoint

UAT: https://test.amwalpg.com:14443/

Production: https://webhook.amwalpg.com/

Endpoint: Execute/PayByToken

Description

This endpoint is used to execute the payment Using CustomerId and CustomerTokenId without all card data as the transaction will be executed as Token transaction.

Headers

	Mandatory
Content-Type	application/json

Request

Sample Request

Request Type: POST

```
{
  "amount": 1,
  "terminalId":380024,
"merchantId":74417,
  "customerId":"fe60f286-b67e-47b2-81db-168d27f97288", //Retrieved From Execute API Response when IsTokenized equal true
  "customerTokenId":"73b76aca-3876-4db9-859c-f56872c779c2",//Retrieved From Execute API Response when IsTokenized equal true
  "requestDateTime": "2023-07-24T15:48:26.101Z",
  "clientMail": "client-email@test.com",
  "currencyCode": "512",
  "transactionId": "fa4d322e-55f6-41db-8b9b-7acf8fcc1390",
  "secureHashValue": "84EB3BF8F62EF25717D1E9E13C3CFB719A890980BBF2631AFD8965182ADE1754"
}
```

Parameters description

Field Name	Mandatory	Field Type	Constraints	Description	Sample Value

customerId	Yes	Guid	Max length: 32 characters	the customer id which already has a registered token in our system.	5ee9205a-b53d4563-8b07f90507c2866d
customerTokenId	Yes	Guid	Max length: 32 characters	the customer Token id which refers to the saved card token for direct payment without the need for card data and will run as MOTO transaction	ea0a60d2-e9be4c35-b15a2a7f031a821f
amount	Yes	Numeric	Length: 1-9	The purchase transaction amount	1
terminalId	Yes	Numeric	Max length: 30 characters	The transaction initiator terminal ID	140052
merchantId	Yes	Numeric	Max length: 30 characters	The transaction initiator merchant ID	1345
merchantReference	No	String	Length: 4	Merchant ref code	1234
requestDateTime	Yes	String	Should be a valid date	The transaction request date, In UTC date time format	2023-03-08T20:51:38.401Z
clientMail	No	String	Max length: 256 characters		demoemail@mail.com
currencyCode	Yes	Numeric	Length:2-4	Code to display with currency	512
transactionId	Yes	String	Max length: 256 characters	Unique id to identify transaction.	5ee9205a-b53d4563-8b07f90507c2866d
secureHashValue	Yes	String	Max length: 256 characters		3DB2F9CB975DADB533A2068F5C2911F9CCFF8AA0DFF3F92

Response

Sample Success Response


```
{
  "success": true,
  "responseCode": "00",
  "message": "Success",
  "data": {
    "systemTraceNr": null,
```

```
    "message": "CAPTURED - ",
    "transactionId": "b81e7509-6ca0-4ab2-89c1-f373bbd4d91b",
    "isOtpRequired": false,
    "hostResponseData": {
      "TransactionId": "202431897300287",
      "Rrn": "431880000100",
      "TrackId": "b81e75096ca04ab289c1f373bbd4d91b",
      "PaymentId": null,
      "Auth": "490908"
    },
    "terminalId": 221143,
    "transactionTypeId": 2,
    "transactionTypeDisplayName": "Purchase",
    "customerId": null,
    "merchantId": 7921,
    "currency": null,
    "amount": 1,
    "currencyId": 512
  },
  "errorList": []
}
```


Success Parameters description

Field Name	Field Type	Description	Sample Value
success	Boolean	Boolean to define the status of the transaction	True|false
responseCode	String	Response code from the API	00
message	String	The response message	Success
data	Object	Object that contains the transaction data	Object properties in the below
data. systemTraceNr	String	System trace number	
data. message	String	Message associated with the transaction result.	“CAPTURED”
data. transactionId	String	The executed transaction number	5ee9205a-b53d-4563-8b07-f90507c2866d

data.isOtpRequired	Boolean	Boolean to define the status whether OTP is required or not	True|false
data. transactionId	String	Transaction identifier	5ee9205a-b53d-4563-8b07-f90507c2866d
data. hostResponseData	Object	Host response data.	
data.hostResponseData. TransactionId	String	Transaction identifier within the host response data.	963cffd7-d586-46c6-8aaf-3b62105dc644
data. hostResponseData .Rrn	String	Retrieval Reference Number	432780000039
data. hostResponseData. TrackId	String	Track identifier within the host response data.	T12345
data.hostResponseData. PaymentId	String	Payment identifier within the host response data.	AM561UORC520Z
terminalId	numeric	Unique identifier for the terminal used.	380024
transactionTypeId	numeric	Transaction Type Id	“2”
transactionTypeDisplayName	String	Transaction display name based on localization header Accept-Language	"بيع" | ”Purchase”
merchantId	String	Merchant Id	7921
currency	String	Currency Name used to execute the transaction	"ريال عما ن" | “OMR”
amount	String	Executed Transaction Amount	
currencyId	Numeric	Currency Id used to execute the transaction	512

customerId	Guid	Customer Id will be Returned when IsTokenized equal to true And merchant is enabled to execute recurring payments This can be used later while executing transaction with PayByToken by providing customerId and customerTokenId without full card data of the customer	89217ad0-872d-45c3a6b3-6cba7967d836
customerTokenId	Guid	Customer Token Id will be Returned when IsTokenized equal to true And merchant is enabled to execute recurring payments This can be used later while executing transaction with PayByToken by providing customerId and customerTokenId without full card data of the customer	89217ad0-872d-45c3a6b3-6cba7967d836
errorList	String []	Should be empty in case of successful response	

Sample Failure Response


```
{
  "success": false,
  "responseCode": "61",
  "message": "APGEX:  62cfc42bff3b",
  "data": null,
  "errorList": [
    "Token Not Found"
  ]
}
```

Failure Parameters description

Field Name	Field Type	Description	Sample Value
success	Boolean	Boolean to define the status of the transaction	false
responseCode	String	The response code from the API	02
message	String	The response message	"Token Not Found"
data	object	Object that contains the transaction data	
errorList	String []	List of errors	["Token Not Found"]